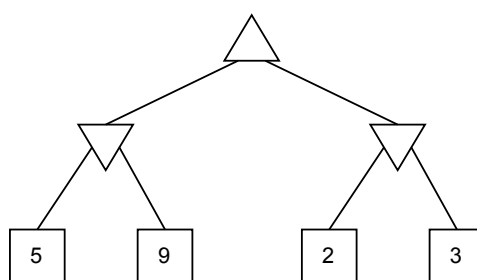




سوالات نظری

سوال ۱:

(آ) نادرست؛ تعداد گره‌های هرس شده، به مقدار اولین گرهی که در آن قسمت باز شده و همچنین به مقدار گره‌های قبلی بستگی دارد. مثال زیر را در نظر بگیرید:



اگر بخواهیم در هر مرحله گره‌ها را از چپ به راست بررسی کنیم، ابتدا بین مقادیر ۵ و ۹، ۵ انتخاب می‌شود. سپس گره ۲ باز شده و چون ۲ از ۵ کمتر است، گره ۳ هرس می‌شود. اما اگر بخواهیم گره‌ها را از راست به چپ باز کنیم، بین ۳ و ۲، ۲ انتخاب شده ولی بعد از باز شدن گره ۹، ۲ $\nless 9$ پس نمی‌توانیم گره آخر را هرس کنیم و باید باز شود.

(ب) نادرست؛ به عنوان نمونه ممکن است پس از کاهش دامنه‌ها با الگوریتم arc consistency (مثل AC3)، یال‌های مسئله سازگار شوند (arc consistency) ولی پاسخی وجود نداشته باشد. مثلاً در مسئله رنگ‌آمیزی نقشه، سه شهر دوه‌دو همسایه را در نظر بگیرید که هر یک فقط دو رنگ آبی و قرمز در دامنه خود دارند. در این حالت arc consistency برقرار است، اما مسئله هیچ پاسخی ندارد. بنابراین همچنان به backtracking نیاز داریم تا این موضوع را تشخیص دهیم.

اما با برقراری سازگاری تعمیم‌یافته (strong n -consistency) می‌توان مسئله را بدون backtracking حل کرد؛ هرچند در این حالت هم همچنان باید مقادیر متغیرها را به ترتیب انتخاب کنیم.

(ج) نادرست: مسائل CSP به طور کلی در دسته مسائل NP-Complete قرار می‌گیرند و در بدترین حالت برای حل کردن آنها نیاز به زمان نمایی داریم. تکنیک‌های MRV و LCV به همراه الگوریتم Arc Consistency می‌توانند در بسیاری از موارد باعث شوند که به طور میانگین، تعداد backtrack ها به شدت کاهش یابد. اما این روش‌ها در بدترین حالت نمی‌توانند مرتبه تعداد backtrack ها را تغییر دهند. (اگر الگوریتم‌ها مشخص باشند، می‌توانیم مثالی بزنیم که تعداد backtrack ها در آن از مرتبه $O(d^n)$ شود).

(د) درست: هنگامی که مسئله CSP درختی باشد، یعنی بین گره‌های مختلف (که همان متغیرهای مسئله هستند) هیچ دوری وجود ندارد و بین دو گره مختلف حداکثر یک مسیر وجود دارد.

در این حالت، الگوریتم به این شکل است:

- یک گره را به دلخواه به عنوان ریشه انتخاب کرده و گره‌ها را با ساختار درختی (از ریشه به برگ) مرتب می‌کنیم. ($O(n)$)
- از برگ‌ها شروع کرده و به سمت ریشه حرکت می‌کنیم. در هر مرحله، بررسی می‌کنیم که آیا به ازای هر مقدار در دامنه گره والد حداقل یک مقدار سازگار در دامنه هر فرزند آن وجود دارد یا خیر. اگر وجود نداشت، آن مقدار را از دامنه والد حذف می‌کنیم. ($O(nd^2)$)
- حال از ریشه شروع کرده، به سمت پایین حرکت می‌کنیم و برای هر گره، یکی از مقادیر سازگار با انتخاب‌های قبلی (والد) را انتخاب می‌کنیم. چون قبلاً سازگاری دامنه‌ها بررسی شده، در این مرحله نیازی به backtrack نداریم و تمام گره‌ها مقداردهی می‌شوند. ($O(nd)$)

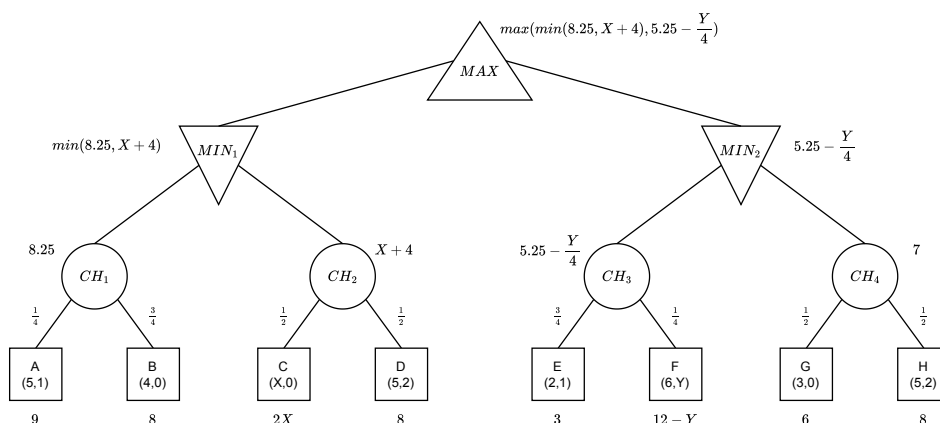
(ه) نادرست؛ می‌دانیم که در مسائل CSP حالت‌های هدف زیرمجموعه‌ای از حالت‌های کامل هستند. الگوریتم BFS، ابتدا تمام حالت‌های partial را بررسی می‌کند تا در نهایت به حالت‌های کامل برسد؛ در حالی که الگوریتم DFS حالت‌های کامل را زودتر بررسی می‌کند. بنابراین BFS در مسائل CSP عملکرد بدتری نسبت به DFS دارد.

همچنین الگوریتم BFS نیاز به فضای ذخیره‌سازی از مرتبه $O(d^n)$ دارد که مقدار بسیار زیادی است.

(و) نادرست؛ ممکن است در یک مسئله ترتیب انتخاب متغیرها و اولویت مقادیری که به آنها می‌دهیم طوری باشد که الگوریتم backtracking به سرعت یکی از پاسخ‌های درست را پیدا کند. در حالی که تلاش برای برقراری strong k -consistency به مدت زمان زیادی نیاز داشته باشد.

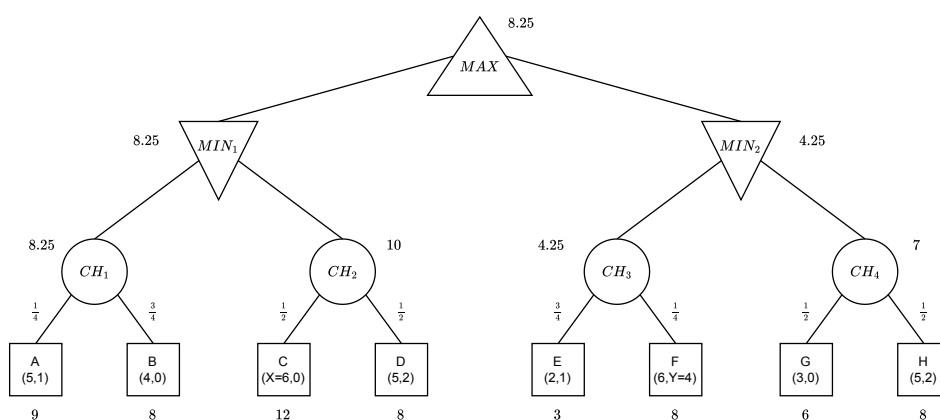
سوال ۲:

(آ) اگر برای تعیین ارزش گره‌های CHANCE، مقدار امید ریاضی ارزش آنها را حساب کنیم، ارزش گره‌ها به این صورت خواهد بود:



توجه داریم که $v(MIN_2) = \min(5.25 - \frac{Y}{4}, 7) = 5.25 - \frac{Y}{4}$

(ب) در این حالت درخت به این شکل خواهد بود:



همان طور که مشاهده می‌شود، $v(MIN_1) = 8.25$ و $v(MIN_2) = 4.25$. بنابراین بازیکن MAX که به دنبال بیشینه کردن امتیاز است، در ریشه، شاخه سمت چپ را انتخاب می‌کند.

(ج) همان طور که در بخش اول محاسبه کردیم:

$$v(MIN_1) = \min(8.25, X + 4)$$

$$v(MIN_2) = 5.25 - \frac{Y}{4}$$

بنابراین اگر بخواهیم بازیکن MAX شاخه سمت راست را انتخاب کند، داریم:

$$\begin{aligned}
 v(MIN_2) > v(MIN_1) &\Rightarrow 5/25 - \frac{Y}{4} > \min(8/25, X + 4) \\
 \xrightarrow{Y \geq 0 \Rightarrow 5/25 - \frac{Y}{4} < 8/25} &X + 4 < 8/25 \quad \wedge \quad X + 4 < 5/25 - \frac{Y}{4} \\
 &\Rightarrow X < 4/25 \quad \wedge \quad X + \frac{Y}{4} < 1/25 \\
 &\Rightarrow (X, Y) \in \{(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 0)\}
 \end{aligned}$$

و اگر بخواهیم ارزش دو شاخه برابر شود:

$$\begin{aligned}
 v(MIN_2) = v(MIN_1) &\Rightarrow 5/25 - \frac{Y}{4} = \min(8/25, X + 4) \\
 &\Rightarrow X + 4 < 8/25 \quad \wedge \quad X + 4 = 5/25 - \frac{Y}{4} \\
 &\Rightarrow X < 4/25 \quad \wedge \quad X + \frac{Y}{4} = 1/25 \\
 &\Rightarrow (X, Y) \in \{(0, 5), (1, 1)\}
 \end{aligned}$$

(د) طبق فرض سوال هرس تنها در گره‌های MIN/MAX انجام می‌شود، بنابراین تا قبل از اینکه گره MAX مقداردهی اولیه نشده باشد، نمی‌توانیم گرهی را هرس کنیم. پس باید صبر کنیم تا گره‌های A, B, C و D دیده شوند. در این حالت:

$$v(CH_1) = 8/25, \quad v(CH_2) = 10, \quad v(MIN_1) = 8/25, \quad v(MAX) = 8/25$$

حال می‌توانیم به زیرگره سمت راست برویم. مشابه قسمت قبل برای اینکه بتوانیم گرهی را هرس کنیم، ابتدا باید MIN_2 مقداردهی شده باشد، پس باید CH_3 مقدار داشته باشد و در نتیجه E و F دیده شده‌اند. در این حالت می‌دانیم که $v(MIN_2) \leq v(CH_3) = 4/25$. از آنجا که $v(MAX) = 8/25$ و $v(MAX) > v(MIN_2)$ ، نیازی به دیدن گره‌های G و H و محاسبه ارزش CH_4 نداریم، چون به هر حال بازیکن MAX مسیر سمت راست را انتخاب نمی‌کند. بنابراین زیردرخت CH_4 (شامل برگ‌های G و H) هرس می‌شود.

سوال ۳:

(آ) در مدل سازی مسئله به صورت CSP هر کارمند را یک متغیر و سه شیفت مورد نظر را دامنه متغیرها در نظر می گیریم:

- متغیرها: سارا (S)، نهال (N)، پریا (P) و زهرا (Z)
- دامنه متغیرها: {صبح (۱)، عصر (۲)، شب (۳)}
- محدودیت ها:

$$C_{SN}: [(S = N \vee S = 3 \vee N = 3)] \wedge [(S \neq N \vee S = N = 2)]$$

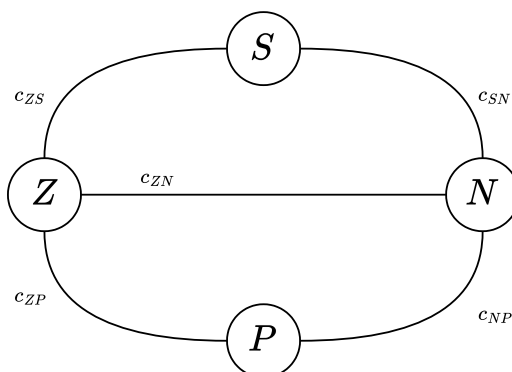
$$C_{ZS}: (Z \neq S) \wedge (Z > S)$$

$$C_{ZN}: Z \neq N$$

$$C_{ZP}: Z \neq P$$

$$C_{NP}: N \neq P$$

گراف شرط های مسئله به این صورت است:



(ب) از الگوریتم AC-3 استفاده می کنیم. یک صف (queue) از تمام یال ها نگه داشته، و در هر مرحله دامنه یک متغیر را با توجه به یال خارج شده به روز کرده و در صورت نیاز، یال هایی را به صف اضافه می کنیم. در هر مرحله یال خارج شده از صف را با رنگ قرمز و یال هایی را که در مرحله قبل به صف اضافه شده اند (در صورت وجود) را با رنگ آبی نمایش می دهیم.

step	Queue	Domains
۱	(S, Z), (Z, S), (N, S), (S, N), (Z, N), (N, Z), (Z, P), (P, Z), (N, P), (P, N)	$D(S) = \{1, 2, \textcolor{red}{3}\}$, $D(N) = \{1, 2, 3\}$, $D(P) = \{1, 2, 3\}$, $D(Z) = \{1, 2, 3\}$
۲	($\textcolor{red}{Z}, S$), (N, S), (S, N), (Z, N), (N, Z), (Z, P), (P, Z), (N, P), (P, N)	$D(S) = \{1, 2\}$, $D(N) = \{1, 2, 3\}$, $D(P) = \{1, 2, 3\}$, $D(Z) = \{\textcolor{red}{1}, 2, 3\}$
۳	($\textcolor{red}{N}, S$), (S, N), (Z, N), (N, Z), (Z, P), (P, Z), (N, P), (P, N), (S, Z)	$D(S) = \{1, 2\}$, $D(N) = \{\textcolor{red}{1}, 2, 3\}$, $D(P) = \{1, 2, 3\}$, $D(Z) = \{2, 3\}$
۴	($S, \textcolor{red}{N}$), (Z, N), (N, Z), (Z, P), (P, Z), (N, P), (P, N), (S, Z)	$D(S) = \{1, 2\}$, $D(N) = \{2, 3\}$, $D(P) = \{1, 2, 3\}$, $D(Z) = \{2, 3\}$

step	Queue	Domains
۵	$(Z, N), (N, Z), (Z, P), (P, Z), (N, P), (P, N), (S, Z)$	$D(S) = \{1, 2\}, D(N) = \{2, 3\}, D(P) = \{1, 2, 3\}, D(Z) = \{2, 3\}$
۶	$(N, Z), (Z, P), (P, Z), (N, P), (P, N), (S, Z)$	$D(S) = \{1, 2\}, D(N) = \{2, 3\}, D(P) = \{1, 2, 3\}, D(Z) = \{2, 3\}$
۷	$(Z, P), (P, Z), (N, P), (P, N), (S, Z)$	$D(S) = \{1, 2\}, D(N) = \{2, 3\}, D(P) = \{1, 2, 3\}, D(Z) = \{2, 3\}$
۸	$(P, Z), (N, P), (P, N), (S, Z)$	$D(S) = \{1, 2\}, D(N) = \{2, 3\}, D(P) = \{1, 2, 3\}, D(Z) = \{2, 3\}$
۹	$(N, P), (P, N), (S, Z)$	$D(S) = \{1, 2\}, D(N) = \{2, 3\}, D(P) = \{1, 2, 3\}, D(Z) = \{2, 3\}$
۱۰	$(P, N), (S, Z)$	$D(S) = \{1, 2\}, D(N) = \{2, 3\}, D(P) = \{1, 2, 3\}, D(Z) = \{2, 3\}$
۱۱	(S, Z)	$D(S) = \{1, 2\}, D(N) = \{2, 3\}, D(P) = \{1, 2, 3\}, D(Z) = \{2, 3\}$
۱۲	$\emptyset$	$D(S) = \{1, 2\}, D(N) = \{2, 3\}, D(P) = \{1, 2, 3\}, D(Z) = \{2, 3\}$

همان طور که مشاهده می‌شود، در انتها دامنه‌ها به این صورت هستند:

$$D(S) = \{1, 2\}$$

$$D(N) = \{2, 3\}$$

$$D(P) = \{1, 2, 3\}$$

$$D(Z) = \{2, 3\}$$

سوال ۴:

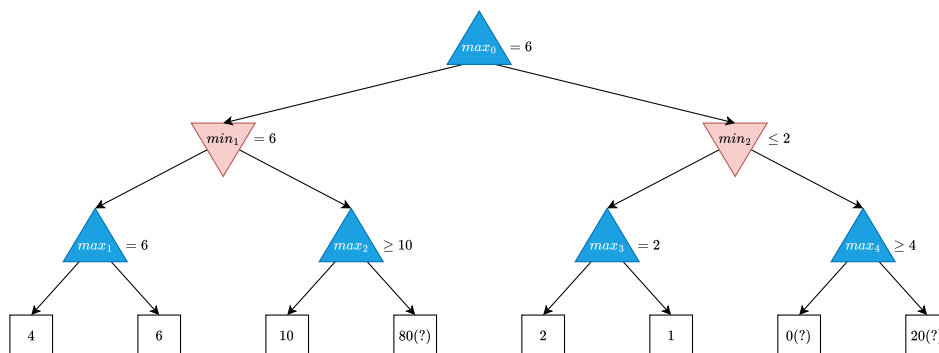
(آ) برای پیشینه کردن تعداد برگ‌های قابل هرس، می‌توانیم به این طریق عمل کنیم: در زیردرخت سمت چپ، باید ابتدا یک مقدار برای min_1 مشخص شده باشد. پس باید دو برگ x_1 و x_2 را دیده باشیم تا مقدار max_1 مشخص شده باشد. اما می‌توان کاری کرد که با دیدن برگ x_3 مطمئن شویم که $max_2 \geq min_1$ و بتوانیم x_4 را هرس کنیم. پس باید:

$$x_3 \geq \max(x_1, x_2)$$

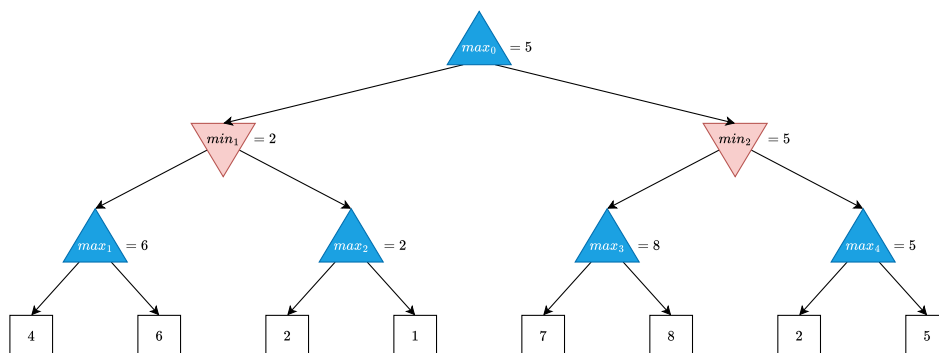
در زیردرخت سمت راست، باید مقدار max_3 مشخص شده باشد تا min_2 مقدار اولیه گرفته باشد. برای اینکه بتوانیم باقی این زیردرخت (برگ‌های x_5 و x_6) را هرس کنیم، باید مقدار max_3 کمتر مساوی مقدار فعلی max (که همان مقدار min_1 است) باشد. چون در این صورت مطمئن هستیم که min_2 مقدار بیشتر از max نمی‌گیرد و بازیکن max شاخه سمت راست را انتخاب نمی‌کند. بنابراین:

$$min_2 \leq \max(x_5, x_6) \leq min_1 = \max(x_1, x_2)$$

با توجه به موارد بالا، می‌توانیم درخت را به این شکل مقداردهی کنیم:



(ب) می‌توان گفت برای این بخش، باید برعکس قسمت قبل عمل کنیم. با استدلال مشابه، در زیردرخت سمت چپ، باید $x_3 < \max(x_1, x_2)$ تا نتوان x_4 را هرس کرد. همچنین باید $\max(x_5, x_6) > min_1$ تا مجبور شویم زیردرخت min_2 را ببینیم. و در نهایت، باید $x_5 < \max(x_5, x_6)$ تا نتوانیم x_6 را هرس کنیم. با توجه به این موارد، می‌توان درخت را به این صورت مقداردهی کرد:



سوال ۵:

(آ) در الگوریتم backtracking، هر گاه تمام حالات ممکن برای یک متغیر بررسی شوند و هیچ یک امکان پذیر نباشد، backtrack کرده و به متغیر قبلی بر می گردیم. در این صورت، روند اجرای الگوریتم برای این مسئله به این صورت می باشد (در هر مرحله، مقداردهی هایی که در آن مرحله انجام شده را با رنگ قرمز، مقدارهای مراحل قبلی را با رنگ آبی مشخص کرده ایم. همچنین مقادیر قبلی که ممکن نبودن را با یک خط روی آنها مشخص کرده ایم):

step	A_1	H	A_4	F_1	A_2	F_2	A_3	T
۱	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B
۲	R, G, B	R , G, B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B
۳	R, G, B	R , G, B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B
۴	R, G, B	R , G, B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B
۵	R, G, B	R , G, B	R, G, B	R, G, B	R , G , B	R, G, B	R, G, B	R, G, B
۶	R, G, B	R , G, B	R, G, B	R, G, B	R , G , B	R, G, B	R, G, B	R, G, B
۷	R, G, B	R , G, B	R, G, B	R, G, B	R , G , B	R, G, B	R , G , B	R, G, B
۸	R, G, B	R , G, B	R, G, B	R, G, B	R , G , B	R , G, B	R, G, B	R, G, B
۹	R, G, B	R , G, B	R, G, B	R, G, B	R , G , B	R , G, B	R , G , B	R, G, B
۱۰	R, G, B	R , G, B	R, G, B	R, G, B	R , G , B	R , G , B	R, G, B	R, G, B
۱۱	R, G, B	R , G, B	R, G, B	R, G, B	R , G , B	R , G , B	R , G , B	R, G, B
۱۲	R, G, B	R , G, B	R, G, B	R, G, B	R , G , B	R , G , B	R, G, B	R, G, B
۱۳	R, G, B	R , G, B	R, G, B	R, G, B	R , G , B	R, G, B	R, G, B	R, G, B
۱۴	R, G, B	R , G, B	R, G, B	R , G, B	R, G, B	R, G, B	R, G, B	R, G, B
۱۵	R, G, B	R , G, B	R, G, B	R , G, B	R , G , B	R, G, B	R, G, B	R, G, B
...								
۳۴	R, G, B	R , G, B	R , G , B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B
۳۵	R, G, B	R , G, B	R , G , B	R, G, B	R, G, B	R, G, B	R, G, B	R, G, B
۳۶	R, G, B	R , G, B	R , G , B	R, G, B	R , G , B	R, G, B	R, G, B	R, G, B
۳۷	R, G, B	R , G, B	R , G , B	R, G, B	R , G , B	R, G, B	R, G, B	R, G, B
۳۸	R, G, B	R , G, B	R , G , B	R, G, B	R , G , B	R, G, B	R, G, B	R, G, B
۳۹	R, G, B	R , G, B	R , G , B	R, G, B	R , G , B	R, G, B	R, G, B	R , G , B

بنابراین در انتها:

$$A_1 = R, \quad H = G, \quad A_4 = B, \quad F_1 = R, \\ A_2 = B, \quad F_2 = R, \quad A_3 = R, \quad T = B$$

(ب) طبق قاعده MRV، در هر مرحله پس از assign کردن مقدار جدید، برای تمام متغیرها بررسی می‌کنیم که چه مقادیری از دامنه ممکن هستند، مقادیر غیر قابل قبول را خط زده و متغیری را انتخاب می‌کنیم که اعضای کمتری در دامنه آن باقی مانده باشند. در صورت تساوی، طبق DegreeHeuristic متغیری را انتخاب می‌کنیم که "محدودیت‌های بیشتری دارد" (گره متناظر با آن در گراف شروط، یال‌های بیشتری دارد).

step	A_1	H	A_4	F_1	A_2	F_2	A_3	T
۱	$\mathcal{R}, G, B$	$\mathcal{R}, G, B$	$\mathcal{R}, G, B$	R, G, B	$\mathcal{R}, G, B$	R, G, B	$\mathcal{R}, G, B$	$\mathcal{R}, G, B$
۲	$\mathcal{R}, \mathcal{G}, B$	$\mathcal{R}, \mathcal{G}, B$	$\mathcal{R}, G, B$	R, G, B	$\mathcal{R}, \mathcal{G}, B$	R, G, B	$\mathcal{R}, \mathcal{G}, B$	$\mathcal{R}, G, B$
۳	$\mathcal{R}, \mathcal{G}, B$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, G, \mathcal{B}$	R, G, B	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	R, G, B	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, G, B$
۴	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, G, \mathcal{B}$	R, G, B	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	R, G, B	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, G, B$
۵	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	R, G, B	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	R, G, B	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, G, B$
۶	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$R, \mathcal{G}, B$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$R, \mathcal{G}, B$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, B$
۷	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, B$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$R, \mathcal{G}, B$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, B$
۸	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, B$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, B$	$\mathcal{R}, \mathcal{G}, \mathcal{B}$	$\mathcal{R}, \mathcal{G}, B$

می‌بینیم که تعداد گام‌های مورد نیاز برای رسیدن به جواب بسیار کمتر از حالت قبل (الگوریتم backtracking ساده) است. هرچند در هر مرحله نیاز به پردازش اضافه برای چک کردن دامنه هر متغیر داریم. همچنین در این حالت نیازی به backtrack کردن نداشتیم. پاسخ نهایی نیز با پاسخی که در بخش قبل به آن رسیدیم متفاوت است:

$$A_1 = B, \quad H = R, \quad A_4 = G, \quad F_1 = R, \\ A_2 = G, \quad F_2 = R, \quad A_3 = B, \quad T = G$$